# Design System on a Live Enterprise Banking Platform

*Product Design - Design Systems - Enterprise Banking - 2026*

> *A pragmatic, scalable design system built inside a live, Ant Design-based corporate banking product, where adoption and reuse mattered more than theoretical purity.*

## Context

I joined an enterprise corporate banking platform after it was already in production. The product had more than 90 screens, permission-heavy workflows, and institutional users who depended on predictable, low-friction interfaces for operational banking tasks.

The frontend was built on Ant Design before I joined. That decision was already made, business-approved, and not negotiable. The frontend team had also created a custom theme in code from the beginning, with their own token-like naming structure already in place.

The design team was small: two designers total. I owned the design system while also contributing to product design work. There was no dedicated design system engineer, no greenfield rebuild, and no pause in delivery. The system had to layer onto a live product without breaking what was already in production.

From the beginning, I treated the system as infrastructure for a family of internal banking tools, not just a cleanup layer for one product. The platform already had 90+ screens, and the same interaction problems - forms, tables, validation states, permissions, approvals, and operational workflows - were likely to repeat across adjacent tools.

## The Problem

The inconsistency was not caused by one bad pattern. It was the cumulative effect of several months of product decisions made without a shared system: spacing values drifted, component states varied across pages, and color usage changed from screen to screen.

The product owner noticed it. The designers noticed it. But the most important signal came from frontend: developers were stopping repeatedly to ask design questions. On a normal day, we were getting roughly 8 to 9 questions such as:

- Why does this component look different here?
- Which spacing value should we use?
- What is the correct state for this interaction?
- Should this be Ant Design default or custom?

That changed how I framed the work. Inconsistency was not only a design quality issue. It was slowing product delivery. Developers stopping 8-9 times a day for visual decisions meant the product team did not have a shared operating language.

## My Role

I owned the design system end to end: structure, tokens, component scope, maintenance, documentation, and evolution. I made independent decisions on naming, token structure, and what should or should not become part of the system.

For anything that required frontend effort beyond Ant Design defaults, I coordinated directly with the frontend tech lead. We reviewed the need, estimated the effort, and decided whether it belonged in the current sprint or should be tracked as backlog or technical debt. Nothing changed in the component layer without that conversation.

## Constraints

- Ant Design was inherited, not chosen. Its component API, default behavior, and theming structure shaped what could be customized quickly and what required additional engineering effort.
- The frontend theme already existed. Engineering had a working naming structure in code before design system work began. Replacing it mid-project would have created refactor cost without immediate product gain.
- The product was live. The system could not require a reset or redesign of existing screens. It had to improve consistency incrementally.
- Capacity was limited. With a small design team and sprint pressure, every customization had to justify its cost against delivery goals.
- The domain required reliability. Enterprise banking workflows depend on clarity, predictable states, and permission-aware interactions; inconsistency increases both user confusion and implementation risk.
- The system needed to scale. Because the bank had adjacent operational tools with similar patterns, the design system had to support reuse without becoming so abstract that it slowed current delivery.

## Key Decision: Adoption Over Semantic Purity

My first instinct was to build a fully semantic token structure: role-based, intent-driven, and clean from a design system perspective. In principle, that was the correct model.

The problem appeared quickly. The frontend team already had a different naming structure in their custom theme. Design and engineering were talking about the same values with different names, which meant every handoff required translation.

I had two options:

- Option A - protect semantic naming. Ask frontend to realign their theme to match the design-side token model. This was theoretically cleaner, but it would have required an already-stretched team to refactor working code during active delivery.
- Option B - align to the existing frontend structure. Accept a less semantically pure naming model, but create a shared language both teams could use immediately without translation.

I chose Option B. A design system only creates value if it gets adopted. A perfectly semantic model that creates daily friction is not a working system. A pragmatic model that both teams use without thinking is more valuable inside a live product.

This was a real trade-off. The token structure is not as portable as a fully semantic system would be, especially if the platform later needs to support multiple brands or unrelated products. I documented that limitation as intentional debt, not as an accident.

## Governance Model

Because there was no dedicated DS engineer, I treated governance as part of the design work. The goal was to prevent the system from becoming another inconsistent layer on top of the product.

For custom work beyond Ant Design defaults, I used a simple repeatable model:

- Identify the product need or inconsistency.

- Check whether Ant Design default behavior could support it.
- If customization was required, review it with the frontend tech lead.
- Estimate effort and risk.
- Decide whether it belonged in the sprint or should move to backlog or technical debt.
- Document the decision so future screens followed the same logic.

It was not a formal committee or a heavy process. It was lightweight enough to survive sprint pressure, but consistent enough that both designers and engineers understood how component decisions were made.

## How the System Changed the Work

The design system did not try to replace Ant Design. It created a product-specific layer on top of it: shared tokens, agreed component behavior, documented states, and clearer decisions around what should remain default versus what needed product-specific customization.

This mattered because the platform was large and operational. Across 90+ screens, even small inconsistencies become expensive: each new screen can either reuse a known pattern or restart a design-engineering negotiation. The system turned repeated one-off questions into reusable decisions.

## Scalability and Reuse

I did not design the system only to normalize the screens that already existed. I designed it to scale across recurring banking workflows: dense tables, form-heavy flows, validation feedback, permission-aware states, confirmation steps, and operational screens where employees needed to move quickly without guessing.

That meant keeping the system practical rather than overly abstract. I focused on patterns that were already appearing repeatedly or were likely to appear again in adjacent tools. The goal was not to create a theoretical component library. It was to create a reusable product foundation that another team or another banking workflow could adopt without starting from zero.

This mattered later. When a forced-timeline batch transfer project arrived, the team could build from the existing system instead of designing every screen, state, spacing rule, and component behavior from scratch. The system reduced the cost of starting a new flow because core decisions had already been made, tested in production, and shared with frontend.

For me, that was the clearest scalability proof: the design system did not only improve consistency inside the original product. It created reusable infrastructure that helped the bank move faster on a separate operational workflow under pressure.

## Outcome

The clearest outcome was a change in frontend communication. Daily design-related interruptions dropped from roughly 8-9 per day to 3-4, and continued decreasing as component coverage grew.

This was not measured through a formal audit or instrumentation, so I would not present it as a precise metric. It was an observed operational signal. But the change was visible in the nature of the conversations:

- Before: Why does this look different here?
- After: We need a new component or variant for this case.

That shift mattered. The team moved from debating repeated visual inconsistencies to discussing reusable system extensions. Frontend adopted the system without pushback, and the product owner saw visible consistency improvements across screens.

A later outcome was reuse. The same foundation supported a separate batch transfer redesign under a forced timeline. I would not present that as a quantified metric, but it is an important product signal: the system was scalable enough to reduce setup time and decision cost when a new banking workflow needed to move quickly.

## What I Would Do Differently

First, I would map the frontend theme before designing the token structure. I went too deep into design-side semantics before fully understanding the naming reality in code. The final decision worked, but earlier technical mapping would have reduced back-and-forth.

Second, I would track the baseline from day one. A simple informal log of DS-related frontend questions would have made the impact more legible to stakeholders. I had the signal, but not enough documentation around it.

Third, I would make the governance model explicit earlier. The process with the frontend tech lead became reliable, but naming it and documenting it sooner would have helped the team understand it as operating infrastructure, not just repeated coordination.

Fourth, I would document scalability more deliberately: component coverage, reuse across flows, and which patterns were intentionally designed as cross-product infrastructure. The reuse existed, but the evidence would be stronger if it had been tracked from the start.

## Why This Case Matters

This project taught me that design systems in enterprise products are rarely clean-room exercises. The real work is not only defining the right components or tokens. It is understanding the technical reality, choosing which compromises improve adoption, and creating enough governance for the system to keep working under delivery pressure.

In this case, the system succeeded because it respected the product that already existed. It did not optimize for theoretical correctness. It optimized for shared language, reduced ambiguity, faster product delivery, and reuse across related banking workflows.